

Attributes reflection

Document #: P3385R2
Date: 2024-12-12
Project: Programming Language C++
Audience: sg7
Reply-to: Aurelien Cassagnes
<aurelien.cassagnes@gmail.com>
Roman Khoroshikh
<rkhoroshikh@bloomberg.net>
Anders Johansson
<ajohansson12@bloomberg.net>

Contents

- 1 Revision history
- 2 Introduction
 - 2.1 Earlier work
 - 2.2 Scope
 - 2.3 Optionality rule
- 3 Proposed Features
 - 3.1 info
 - 3.2 Reflection operator
 - 3.3 Splicers
 - 3.4 Metafunctions
 - 3.5 Queries
- 4 Proposed wording
 - 4.1 Language
 - 4.2 Library
 - 4.3 Feature-test macro
- 5 Feedback
 - 5.1 Poll
 - 5.2 Implementation
- 6 Conclusion

§ 1 Revision history

Since [P3385R1]

- Fix various issues with samples
- Fix wording
- Rebase on 2996R8

Since [P3385R0]

- Rebase on [P2996R7]
- Reword ignorability section to mention set of rules
- Discuss argument clause in info equal operator
- Discuss appertaining to null statement in reflect expression

§ 2 Introduction

Attributes are used to a great extent, and there likely will be new attributes added as the language evolves.

As reflection makes its way into our standard, what is missing is a way for generic code to look into the attributes appertaining to an entity. That is what this proposal aims to tackle by introducing the building blocks.

We expect a number of applications for attribute introspection to happen in the context of code injection [P2237R0], where, for example, one may want to skip over `[[deprecated]]` members. The following example demonstrates skipping over deprecated members:

```
struct User {
    [[deprecated]] std::string name;
    [[deprecated]] std::string country;
    std::string uuidv5;
    std::string countryIsoCode;
};

constexpr std::vector<info> selectNonDeprecated() {
    std::vector<info> liveMembers;
    constexpr auto attributes = attributes_of(^^[[deprecated]]);
    auto keepLive = [&] (info r) {
        if (!std::ranges::any_of(
            attributes_of(^^r),
            [&] (auto meta) { meta == attributes[0]; }
        )) {
            liveMembers.push_back(r);
        }
    }
```

```
};

template for (constexpr auto member : members_of(^User)) {
    keepLive(member);
}
return liveMembers;
}

// Migrated user will no longer support deprecated fields
struct MigratedUser;
define_aggregate(^MigratedUser, selectNonDeprecated());
```

§ 2.1 Earlier work

While collecting feedback on this draft, we were redirected to [P1887R1], a pre existing proposal. In this paper, the author discusses two topics: ‘user defined attributes’ (also in [P2565R0]) and reflection over said attributes. We believe these two topics need not be conflated; both have intrinsic values on their own. We aim to focus this discussion entirely on **standard** attributes reflection. Furthermore the earlier paper has not seen work following the progression of [P2996R7], and so we feel this proposal is in a good place to fill that gap.

§ 2.2 Scope

Attributes are colloquially split into standard and non-standard. This proposal wishes to limit itself to standard attributes (9.12 [dcl.attr]). We feel that since it is up to the implementation to define how to handle non-standard attributes, it would lead to obscure situations that we don’t claim to tackle here.

A fairly simple (admittedly artificial) example can be built as such: Given an implementation supporting a non-standard `[[privacy::no_reflection]]` attributes that suppresses all reflection information appertaining to an entity, we would have a hard time coming up with a self-consistent system of rules to start with.

Henceforth, in this proposal, ‘attributes’, ‘standard attributes’ are all meant to be equivalent terms for attributes described by the standard.

§ 2.3 Optionality rule

There is a long standing and confusing discussion around the ignorability of attributes. We’ll refer the reader to [P2552R3] for an at-length discussion of this problem, and especially with regard to what ‘ignorability’ really means. This proposal agrees with the discussion carried on there and in [CWG2538]. We also feel that whether an implementation decides to semantically ignore a standard attribute should not matter.

Another interesting conversation takes place in [P3254R0], around the `[[no_unique_address]]` case, which serves again to illustrate that the tension around so called ignorability should not be considered a novel feature of this proposal.

What matters more is the following set of rules that aim to discuss self-consistency

- We should be able to statically reflect on appertaining attributes even if ignored: Implementations are required to parse and check syntactic rules related to attributes. Whether they are ignored semantically later on should not mean that they are removed from the source code during translation¹.
- Splicing attributes on a declaration should be undistinguishable from appertaining those attributes to this declaration.

§ 3 Proposed Features

We put ourselves in the context of [P2996R7] for this proposal to be more illustrative in terms of what is being proposed.

§ 3.1 info

We propose that attributes be a supported *reflectable* property of the expression that is reflected upon. That means value of type `std::meta::info` should be able to represent an attribute in addition to the currently supported set.

§ 3.2 Reflection operator

The current proposition for reflection operator grammar does not cover attributes, i.e., the expression `^^[[deprecated]]` is ill-formed. Our proposal advocates to support expression as following:

```
constexpr auto keepAttribute = ^^[[nodiscard]];
```

The resulting value is a reflection value embedding salient property of the *attribute* which are the token and the argument clause if any. If the *attribute* is not a standard attribute, the expression is ill-formed.

§ 3.3 Splicers

We propose that the syntax

```
[[ [: r : ] ]]
```

be supported in any context where attributes are allowed.

- `[[ [: r : ] ]]` produces a potentially empty *attribute-list* corresponding to the attributes found via reflection `r`. In effect every attributes that are found via `attributes_of(r)` are expanded in place inside `[[ ]]`.

Note that as it stands now *attribute-list* (9.12.1 [dcl.attr.grammar]) does not cover `alignas`. We understand that this limits potential use of the current proposal but also comes with difficulty, so this shall be discussed in a separate paper.

A simple example of splicer using expansion statements is as follows. We create an augmented `enum` introducing a `begin` and `last` enumerator while preserving the original attributes:

```
[[nodiscard]]
enum class ErrorCode {
    warn,
    fatal,
};

[[ [[: ^^ErrorCode :] ]]
enum class ClosedErrorCode {
    begin,
    template for (constexpr auto e : enumerators_of(^^ErrorCode)) {
        return [[:e:]],
    }
    last,
};
```

If the attributes produced through introspection violate the rules of what attributes can appertain to what entity, the program is ill-formed, as usual.

§ 3.3.1 Attribute using

It is worth pointing out the interaction between *attribute-using-prefix* and splice expression that could lead to unexpected results, such as in the following example

```
auto attribute = ^^[nodiscard]];
[[ using CC: debug, [[: attribute :] ]] enum class Code {};
```

While it is unlikely the user intends for the standard attributes to be targeted by `using CC`, current grammar says that the prefix applies to attributes as they are found in the *subsequent* list. To remediate this we can either enforce that *splice-name-qualifier* precedes *attribute-using-prefix* or have those constructs be mutually exclusive as they occur in `[[ [ ] ]]`.

To reduce the need to memorize unintuitive rules, we favor the later of those options, as following:

```
auto attribute = ^^[nodiscard]];
[[ [[: attribute :] ]][[ using CC: debug ]] enum class Code {};
```

§ 3.4 Metafunctions

We propose to add two metafunctions to what has already been discussed in [\[P2996R7\]](#). In addition, we will add support for attributes in the other metafunctions, when it makes

sense.

§ 3.4.1 attributes_of

```
namespace std::meta {  
    consteval auto attributes_of(info entity) -> vector<info>;  
}
```

This would return a vector of reflections representing individual attributes that were appertaining to reflected upon entity.

§ 3.4.2 is_attribute

```
namespace std::meta {  
    consteval auto is_attribute(info entity) -> bool;  
}
```

This would return true if the entity reflection represents a `[ [ attribute ] ]` such as described in `[dcl.attr]`, otherwise, it would return false.

§ 3.4.3 identifier_of, display_identifier_of

Given a reflection `r` designating a standard attribute, `identifier_of(r)` (resp. `u8identifier_of(r)`) should return a `string_view` (resp. `u8string_view`) corresponding to the attribute-token. We do not think the leading `[ [` and closing `]` are meaningful, besides, they contribute visual noise.

A sample follows

```
[[nodiscard]] int func();  
constexpr auto nodiscard = attributes_of(^func);  
    static_assert(identifier_of(nodiscard[0]) ==  
    identifier_of(^[[nodiscard]]));  
static_assert(identifier_of(nodiscard[0]) == "nodiscard"); // != "  
    [[nodiscard]]"
```

Given a reflection `r` that designates an individual attribute, `display_identifier_of(r)` (resp. `u8display_identifier_of(r)`) returns an unspecified non-empty `string_view` (resp. `u8string_view`). Implementations are encouraged to produce text that is helpful in identifying the reflected attribute for display purpose. In the preceding example we could imagine printing `[[nodiscard]]` instead of `discard` as it might be more striking for display purpose to the user.

§ 3.4.4 data_member_spec, define_aggregate

As it stands now, `define_aggregate` allows piecewise building of a class via `data_member_spec`. However, to support attributes pertaining to those data members, we'll need to augment `data_member_options` to encode attributes we may want to attach to a data member.

The structure will change thusly:

```
namespace std::meta {
    struct data_member_options {
        struct name_type {
            template <typename T> requires constructible_from<u8string, T>
            consteval name_type(T &&);

            template <typename T> requires constructible_from<string, T>
            consteval name_type(T &&);
        };

        optional<name_type> name;
        optional<int> alignment;
        optional<int> bit_width;
-       bool no_unique_address = false;
+       vector<info> attributes;
    };
}
```

From there building an aggregate piecewise proceeds as usual

```
struct Empty {};
struct [[nodiscard]] S;
define_aggregate(^S, {
    data_member_spec(^int, {.name = "i"}),
    data_member_spec(^Empty, {.name = "e",
                             .attributes = {^[no_unique_address]]})
});

// Equivalent to
// struct [[nodiscard]] S {
//     int i;
//     [[no_unique_address]] struct Empty { } e;
// };
```

Note here that [P2996R7] includes `no_unique_address` and `alignment` as part of `data_member_options` API. We think that this approach scale poorly in that every new attributes introduced into the standard will lead to discussions on whether or not they deserve to be included in `data_member_options`. There is also little explanations as to why those were picked among all. Attaching attributes through above approach is more in line with the philosophy of leveraging `info` as the opaque vehicle to carry every and all reflections.

We will not pursue this change here but in a follow-up paper, as we wish to keep the scope of change rather small in our proposal.

§ 3.4.5 Other metafunctions

For any reflection where `is_attribute` returns `true`, other metafunctions not listed above are not considered constant expressions

§ 3.5 Queries

We do not think it is necessary to introduce any additional query or queries at this point. We would especially not recommend introducing a dedicated query per attribute (e.g., `is_deprecated`, `is_nouniqueaddress`, etc.). Having said that, we feel those should be achievable via concepts, something akin to:

```
auto attributes = attributes_of(^^[[deprecated]]);

template<class T>
concept IsDeprecated = std::ranges::any_of(
    attributes_of(^^T),
    [deprecatedAttribute] (auto meta) { meta == attributes[0]; }
);
```

§ 4 Proposed wording

§ 4.1 Language

§ 6.8.2 [basic.fundamental] Fundamental types

Augment the description of `std::meta::info` found in new paragraph 17.1 to add standard attribute as a valid representation to the current enumerated list

17-1 A value of type `std::meta::info` is called a *reflection*. There exists a unique *null reflection*; every other reflection is a representation of

...

- an attribute described in this document

- an implementation-defined construct not otherwise specified by this document.

Update 17.2 Recommended practices

17-2 *Recommended practice*: Implementations are discouraged from representing any constructs described by this document that are not explicitly enumerated in the list above (e.g., partial template specializations, ~~attributes~~, placeholder types, statements).

§ 7.6.2.10* [expr.reflect] The reflection operator

Edit ^^ operator grammar to allow reflecting over an attribute

reflect-expression:

```
^^ ::  
^^ unqualified-id  
^^ qualified-id  
^^ type-id  
^^ pack-index-expression  
^^ [ [ attribute ] ]
```

Add the following paragraph at the bottom of 7.6.2.10

- 8 A *reflect-expression* having the form ^^ [[*attribute*]] computes a reflection of the attribute 9.12 [dcl.attr]. If *attribute* is not described in 9.12 [dcl.attr], the behavior is implementation defined.

§ 7.6.10 [expr.eq] Equality Operators

Update new paragraph between 7.6.10 [expr.eq]/5 and /6 to add a clause for comparing reflection of attributes and renumber accordingly

If both operands are of type std::meta::info, comparison is defined as follows:

- (*7) - Otherwise, if one operand represents a *base-specifier*, then they compare equal if and only if the other operand represents the same *base-specifier*.
- (*) - Otherwise if one operand represents an attribute described in this document, then they compare equal if and only if the other operand represents an attribute as well and both those attributes share the same *identifier*.
- (*8) - Otherwise, both operands 0_1 and 0_2 represent descriptions of declarations of non-static data members: Let C_1 and C_2 be invented class types such that each C_k has a single non-static data member having the properties described by 0_k . The operands compare equal if and only if the data members of C_1 and C_2 would be declared with the same type or *typedef-name*, name (if any), *alignment-specifiers* (if any), width, and attributes

§ 9.12.1 [dcl.attr.grammar] Attribute syntax and semantics

Change the grammar to allow *splice-specifier* inside [[]]

```
...  
attribute-specifier:  
[ [ attribute-using-prefixopt attribute-list ] ]  
[ [ splice-specifier ] ]
```

Modify the paragraph 5 to relax appertenance when an attribute is the operand of a reflection expression

- 5 Outside a *reflect-expression*, each~~Each~~ *attribute-specifier-seq* is said to appertain to some entity or statement, identified by the syntactic context where it appears (Clause 8, Clause 9, 9.3). If an *attribute-specifier-seq* that appertains to some entity or statement contains an *attribute* or *alignment-specifier* that is not allowed to apply to that entity or statement, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration (11.8.4), that declaration shall be a definition.

Modify the paragraph 7 to allow consecutive left square brackets following the reflection operator

- 7 Two consecutive left square bracket tokens shall appear only when introducing an *attribute-specifier* ~~or~~, within the *balanced-token-seq* of an *attribute-argument-clause* or as an operand of a *reflect-expression*.

Add the following paragraph

- 8 If an *attribute-specifier* contains a *splice-specifier*, every attribute contained in that reflection appertain to the entity to which that *attribute-specifier* appertain.

[Example 1:

```
struct [[nodiscard, maybe_unused]] Foo;
struct [[ [: ^^Foo :] ]] Bar; // same as struct [[nodiscard,
maybe_unused]] Bar;
```

— end example]

§ 4.2 Library

§ 4.2.1 [meta.reflection.synop] Header **<meta>** synopsis

Add to the [meta.reflection.queries] section from the synopsis, the two metafunctions `is_attribute` and `attributes_of`

```
...
// [meta.reflection.queries], reflection queries
...
constexpr bool is_attribute(info r);
constexpr vector<info> attributes_of(info r);
```

§ 4.2.2 [meta.reflection.queries], Reflection queries

42 `constexpr bool is_attribute(info r);`

Returns: `true` if `r` represents a standard attribute. Otherwise, `false`.

43 `constexpr vector<info> attributes_of(info r);`

Returns: A vector containing reflections of all attributes of the entity represented by `r`

§ 4.2.3 [meta.reflection.names], Reflection names and locations

Update description of `has_identifier` return value to be `true` for reflected attribute and renumber accordingly

`constexpr bool has_identifier(info r);`

1 *Returns:*

...

(1.*) Otherwise, if `r` represents an attribute, then `true`

(1.10) Otherwise `false`

Add a paragraph to `identifier_of` to describe return value of reflected attribute

`constexpr string_view identifier_of(info r);`

4 *Returns:*

...

(4.6) Otherwise, if `r` represents an attribute, then the `attribute-token`

§ 4.3 Feature-test macro

The attribute reflection features is guarded behind macro, augment 15.11 [cpp.predefined]

`__cpp_impl_reflection 2024XXL`
`__cpp_impl_reflection_attributes 2024XXL`

§ 5 Feedback

§ 5.1 Poll

§ 5.1.1 P3385R1: SG7, Nov 2024, WG21 meetings in Wroclaw

Poll: P3385R1: SG7 encourages more work on reflection of attributes as described in the paper.

§ 5.2 Implementation

Features proposed here were implemented on a public fork (off the Bloomberg P2996 branch) of Clang. Although it isn't production ready code it demonstrates the feasibility of the proposal.

§ 6 Conclusion

Originally the idea of introducing a `declattr(Expression)` keyword seemed the most straightforward approach to tackling this problem. However based on feedback, the concern of introspecting on expression attributes was a topic that belongs with the Reflection SG. The current proposal shifted away from the original `declattr` idea to align better with the reflection toolbox. Note also that, as we advocate here for `[[[:r:]]]` to be supported, we recover the ease of use that we first envisioned `declattr` to have.

§ 7 References

- [CWG2538] Jens Maurer. 2021-12-02. Can standard attributes be syntactically ignored?
<https://wg21.link/cwg2538>
- [P1887R1] Corentin Jabot. 2020-01-13. Reflection on attributes.
<https://wg21.link/p1887r1>
- [P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
<https://wg21.link/p2237r0>
- [P2552R3] Timur Doumler. 2023-06-14. On the ignorability of standard attributes.
<https://wg21.link/p2552r3>
- [P2565R0] Bret Brown. 2022-03-16. Supporting User-Defined Attributes.
<https://wg21.link/p2565r0>
- [P2996R7] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-10-13. Reflection for C++26.
<https://wg21.link/p2996r7>
- [P3254R0] Brian Bi. 2024-05-22. Reserve identifiers preceded by `@` for non-ignorable annotation tokens.
<https://wg21.link/p3254r0>
- [P3385R0] Aurelien Cassagnes, Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. 2024-09-16. Attributes reflection.
<https://wg21.link/p3385r0>
- [P3385R1] Aurelien Cassagnes, Roman Khoroshikh, Anders Johansson. 2024-10-15. Attributes reflection.
<https://wg21.link/p3385r1>

1. Discussion in <https://eel.is/c++draft/dcl.attr#depend-2>, imply that translation units need to carry their attributes unchanged to observe that rule. ↩